CS 380: Math for AI; Homework #3: PCA

Ben Belden

October 28, 2026

Due Monday, October 26 at 11:59pm Central

Please read the following independent completion attestation before starting the homework. Then digitally sign it, acknowledging that you understand the guidelines for this homework, and have followed them.

```
I declare that I have completed this assignment entirely on my own,
and following the generative AI policies.
Googling for documentation (e.g., NumPy SVD) and other basic material (e.g., Wikipedia SVD) is fine,
but no use of tools like ChatGPT, either to write code or to write text.
I have read the UAB Academic Integrity Code and understand that any breach
of this code may result in severe penalties, including failure of the class.
```

Signature: *Ben Belden*

Q1: An efficient algorithm to multiply outer products (written)

Give an $O(n^2)$ algorithm for computing $C = (xy^t)^k$ where x and y are n-vectors.

$$(\begin{pmatrix} x_1 \\ x_2 \\ ... \\ x_n \end{pmatrix} \begin{pmatrix} y_1 & y_2 & ... & y_n \end{pmatrix})^k$$

```
outer_k(x, y, k):
    if x.shape[0] != y.shape[1]:
        print(error)
        Return None
    else:
        n = x.shape[0]

    dot = 0
    for pos in range(n):
        product = y[0, pos] * x[pos, 0]
        dot += product

    matrix = np.zeros((n, n))
    for i in range(n):
        for j in range(n):
            matrix[i, j] = x[i, 0] * y[0, j]

    if k == 1:
        return matrix
    else:
        scalar = dot^(k-1)
        return scalar * matrix

outer_k(x, y^t, k)
```

Explain why it is quadratic, and compare it to the naive algorithm.

$C = (xy^t)^k \rightarrow$ O(n^2):

- finding 'n', checking that the dimensions of the two vectors correspond: O(1)
- initialize the stored dot product variable (dot = 0): O(1)
- computing $y^t x$ $(x \cdot y == y \cdot x)$: O(n)
- create a np.zeros((n, n)) variable: O(n^2)
- nested loops to find the outer product between x and y^t $((xy^t)^1)$: O(n^2)
- *depends on implementation - calculate scalar = dot^{k-1}:
  using exponentiation by squaring: O(log k)
  using repeated scalar multiplication: O(k)
- *depends on implementation - calculate scalar $*$ $(xy^t)^1$: O(n^2)

between all of the above operations, the upper-bound time complexity is O(n^2) $\rightarrow$ (nested loops)

The naive approach computes the outer product of vectors (x) and (y), which takes ($O(n^2)$) time. After this computation, the algorithm performs (k - 1) matrix multiplications to calculate ($C = (xy^t)^k$). Each multiplication of two (n×n) matrices takes ($O(n^3)$) time using the standard matrix multiplication algorithm. Therefore, the total time complexity of the naive approach is $O(n^3)$.

The optimized approach uses a scalar dot product ($y^t x$) that takes O(n) time to compute, and raising it to the (k - 1)th power takes O(log k) time using exponentiation by squaring or O(k) time with repeated multiplication. The final scalar multiplication of the outer product takes $O(n^2)$ time.

The main improvement comes from replacing repeated matrix multiplications, which cost ($O(n^3)$) time each, with a dot product and scalar exponentiation or repeated multiplication. The optimized algorithm only needs to build and scale one outer product matrix.

Q2: Axis of rotation (written)

Give an algorithm to compute the axis of rotation of a rotation in 3-space from its rotation matrix.
That is, the input is a rotation matrix of shape (3,3) representing a rotation in 3-space,
and the output is the axis of rotation of this rotation.
Euler proved that any rotation in 3-space is equivalent to a rotation about some axis.

```
get_axis_of_rotation(R):
    compute eigenvalues of R
    if one of eigenvalues of R is 1:
        compute eigenvector of R with eigenvalue == 1
    if no eigenvalues of R are 1:
        compute eigenvector of R with eigenvalue ≈ 1
    normalize the vector
    return vector
```

Explain why you think this algorithm is correct.

"When you rotate an object around an axis, points lying on that axis do not move."

Think of a spinning globe. The globe has an axis of rotation. You decide to draw points along this axis 1 unit apart with the largest point along the axis being the point representing the 'North Pole', the smallest point along the axis representing the 'South Pole', and the origin being the point in which the equatorial plane and the axis of rotation intersect. While the globe is spinning, every point on the surface of the globe moves in a circular motion around the center axis. If you pick a point on the surface of the globe at the same height as a point on the center axis relative to the equatorial plane, and this point is a closer point than a hypothetical previously analyzed point, the smaller the circle of rotation. The distance from the point lying on the center axis from this near point on the surface is shorter (smaller radius). At the exact line of the center axis, the radius of the circle is zero → points on the axis of rotation have zero rotation. These points remain still, and the globe rotates around them. Points along this center axis do not move!

The center axis is the output looking for from the algorithm. Because points along the center axis do not move as the rotation performs, this axis does not change when the rotation is applied. The axis can be seen as a vector. Thus, R transforms the axis vector into itself. How do we find vectors that are not affected whatsoever from a linear transformation? Eigenvalues and eigenvectors.

Looking for a rotation matrix R when applied to a vector v outputs the vector v: $R(v) = v$

Definition of an eigenvector from linear algebra: $Ax = \lambda x$
Eigenvector definition w/ relationship to problem: $Rv = \lambda v$
When the eigenvalue is one: $Rv = v$
We are looking for an eigenvalue from the rotation matrix of size one, the eigenvector corresponding to the eigenvalue is the axis of rotation!

PS: I do not know if an axis of rotation is required to have magnitude one, so my current pseudocode does not consist of normalizing the eigenvector with corresponding eigenvalue 1.
PSS: I decided to normalize the eigenvector ...

Q3: Best-fitting plane (Python)

Given a point cloud in 3-space, build its **best-fitting plane** using PCA.
The input is a PLY file of a point cloud in 3-space.
The output is a point and unit normal of the best-fitting plane.

Please choose the mean as the point on the plane, to simplify testing.
You are to build two implementations.
The first implementation should use spectral analysis.
The second implementation should use singular value decomposition (SVD).

I have given a code template in hw3_26fa380.py.
The point cloud will be read from an input file containing the points, one per line (read_cloud). You should have a separate function for reading the cloud and for mean normalizing the cloud (mean_normalize).

Q4: Testing your best-fitting plane code (Python)

Test your code in the following two ways.

First, test that the two implementations yield (approximately) the same answer.
Note that both implementations use the mean as the point, so they should yield (approximately) the same answer, except the direction of the unit normal is a degree of freedom.

Second, test your code on ground truth datasets: random planar point clouds in 3-space with known (point, normal) representations. A ground truth dataset is a dataset for which you know the correct answer.

This second testing method will require a function to build a ground truth dataset.
Write code to build, given a point and normal, a random point cloud in 3-space on the plane defined by this point and normal.
I have provided a stub for this function (build_random_planar_cloud).

Testing against this type of ground-truth dataset is still a bit tricky.
In particular, the non-uniqueness of the point-normal representation causes some challenges in testing.
Develop a technique to test whether two point-normal representations are equivalent, and use it to test your code against the ground truth datasets.

`np.allclose()`

Rather if the output PC1 corresponding to: the smallest eigenvalue from the covariance matrix eigenvector (spectral), or the last (first) column in the U matrix (SVD) is positive or negative does not matter as long as the digits are the same. The PC1 outputted from the spectral technique and the SVD technique represents the normal of the best fitting plane (2D subspace where data is most spread out, variance) of data from a .ply file. As the output only requires one principal component, the orientation of the plane shall be bypassed. If the PC1 normal vector points up (positive) or down (negative) and the point on the plane is consistent between the two techniques, the point, normal representation describes the same plane.

For output check →
`if (np.allclose(mean1, mean2)): good!`
`if (np.allclose(abs(normal1), abs(normal2))): good!`

If we were needing, wanting to get more precise with multiple principal components, orientation will be taken into great account (I believe).

I checked that both my spectral and SVD techniques to compute the normal vector representing the PC1 from the data through manual work → frogPCA.
frogPCA → a file showcasing a case I formulated to validate the algorithms, six frogs w/ differing three genes each plotted within a 3D space, through the output in stdout of the point, normal from the frog_data file in which prior I knew the correct point, normal correspond to this output from spectral and SVD → confirming validity.
*see frogPCA in notes directory

The file SVDdecomposition.py is a manual implementation of SVD for understanding purposes -> in hw3.belden.26fa380.py, I use the numpy .svd() function.

Robustness?

Ungraded challenges

- compute the best-fitting line in 2-space
- compute the best-fitting ellipse in 2-space
- compute the best-fitting ellipsoid in 3-space

Yet again, no time :(